
<Travlr Getaways>

**CS 465 Project Software Design Document**

Version 1.0

**Table of Contents**



**Document Revision History**

| Version | Date | Author | Comments |
|---|---|---|---|
| 1.0 | <09/20/2024> | <merlin martinez > | <**Executive Summary, Design Constraints, System Architecture View.** > |

**Instructions**

**Executive Summary**

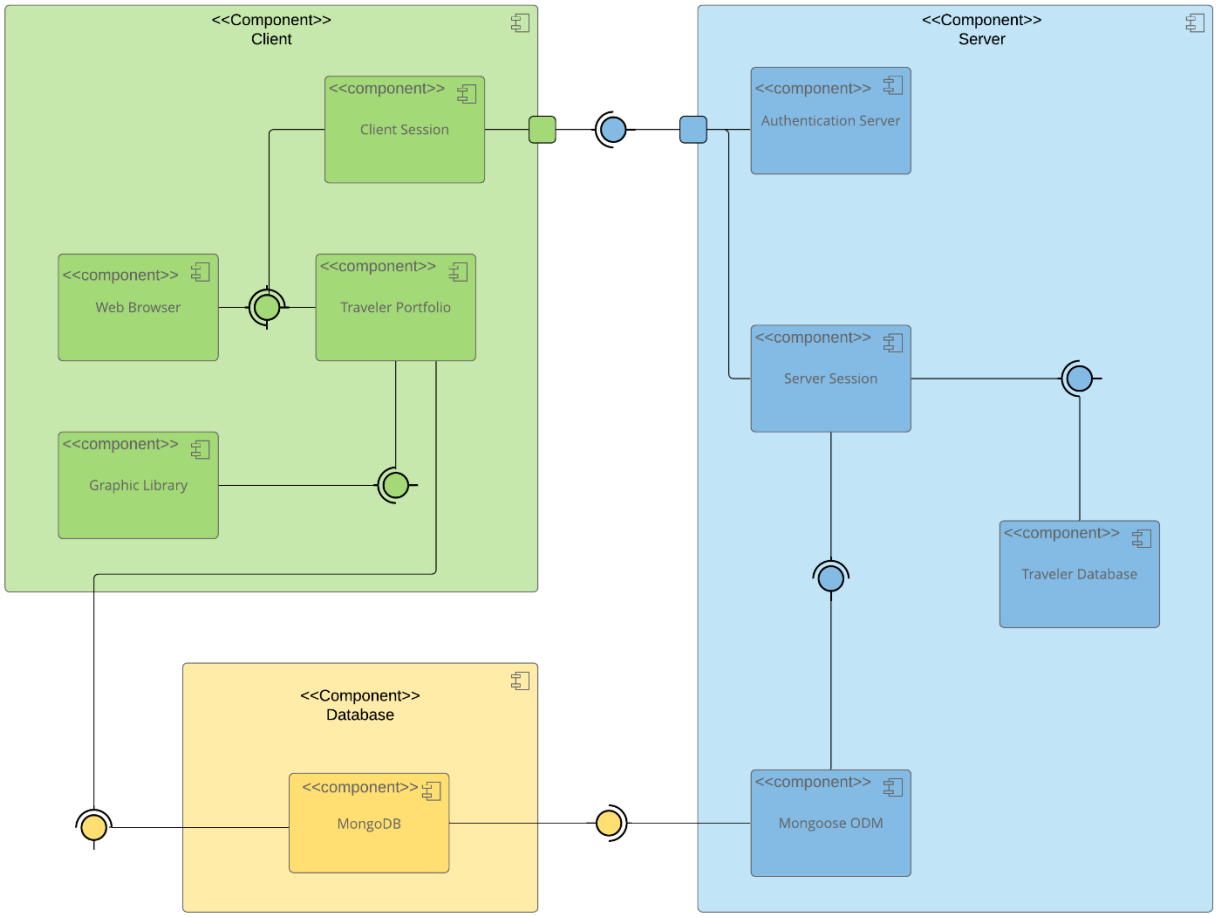
<I'm developing a web app for Travlr Getaways and the web app uses MEAN stack for development and used mongodb as data base and node.js. The front end of the website uses angular.js and then the back end uses xpress.js and Node.js. the main objective of this website is for users to make bookings, buy travel packages and etc. the back end of the website offers a SPA, this means that the web app even if provides a high experience for the customers it might not be the same for the web manager because they might experience data loss of network connectivity. >

**Design Constraints**

<The design constraints for developing the web-based Travlr Getaways application has many steps first there's must be a budget, a understanding on what the client wants how they want their website to look like or as well after the website is done who's going to maintain it, and as well the right personnel needs to be hired like for example someone in cyber security that is able to test and ensure that the system cant be hacked or any info its being stolen, backed end and front end developer, after all this requirement are meet we need to think who are going to be the targeted audience and have a nice appealing design, it needs to be something user friendly that client wont have issues with and lastly maintenance continuous maintenance to prevent hack, injections and as well to make updates and patches so the system can keep running smooth for users. >

**System Architecture View**

**Component Diagram**

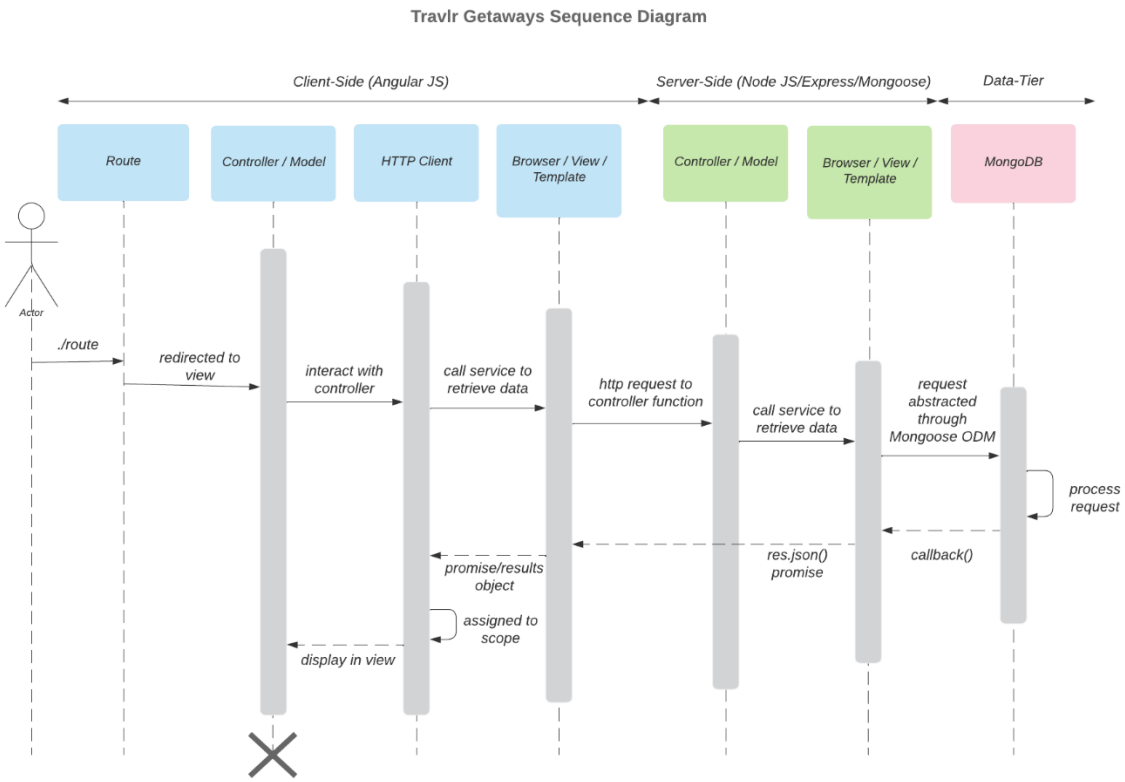



A text version of the component diagram is available: [CS 465 Full Stack Component Diagram Text Version].

<the main components of a website architecture are going to be the following database in this case we are using mongodb which is responsible for storing data like for example users ID, hash, queries etc. the authentication server is in charger pf identifying and as well verifying the users through log in, the server session is responsible for managing the server side for the user's data. Then next we have the server side and client side which are connected by a port. Client session this means that the user information is maintained like for example their preferences and data. Web browser this one allows the user to interact with the system like the server to send and receive requests.

In conclusion the client components are going to be the front end and the angular in MEAN this means that the users can connect and use both the database and the server to display and store data, next the server components contain the Express in MEAN and the node.js files which contains all the require code to make the website work. >

**Sequence Diagram**

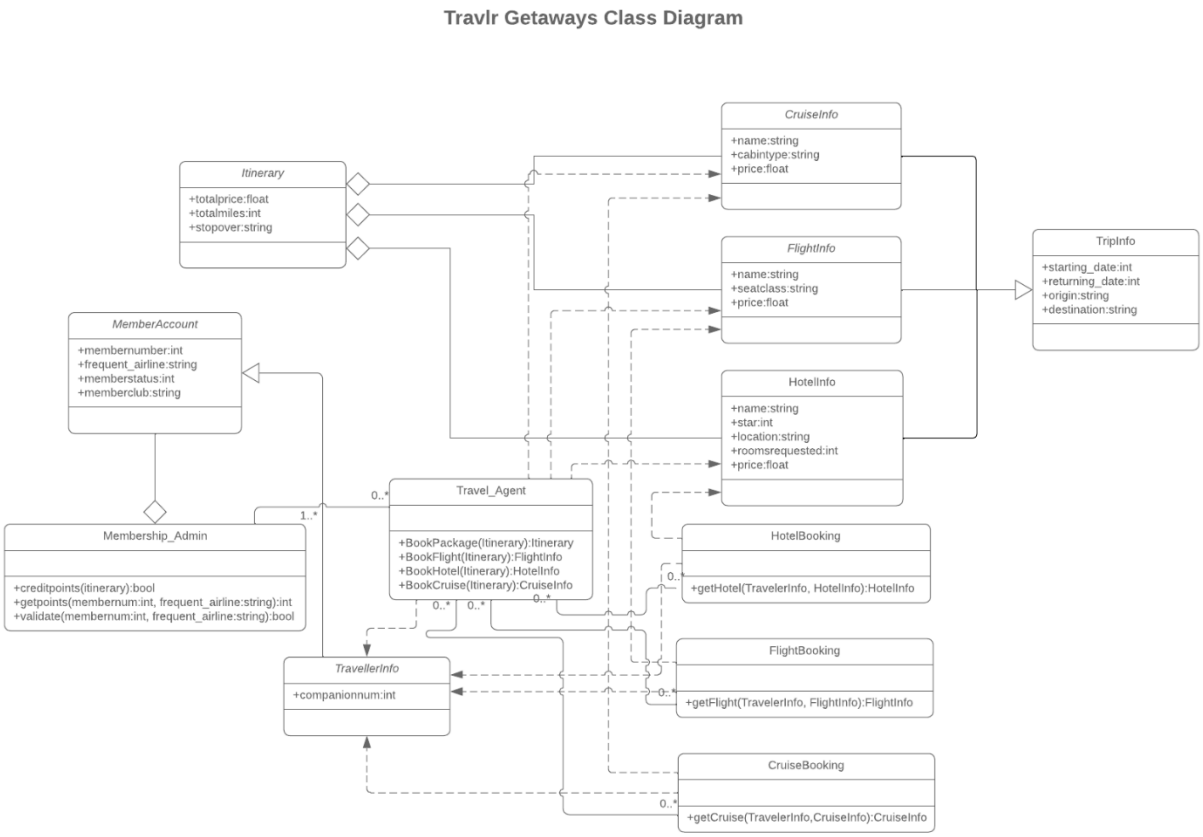



<travlr Getaways is a website for people who want to book and look at trips this means that when a user's interacts with the site, they are initiating a specific action like for example they are able to sign in if they already have an account, viewing/booking trips or performing admin interactions. By users doing this action they are initiating the front-end router, which shows the users' relevant view or template.

A good example is when the user tries to sign in, the router at the front end navigates them to the sign-in page. The controllers are activated this takes care of showing the users the populating the template and generating the view that will be displayed to the user. In addition, the front end communicates with HTTP in which this will trigger the API. Now the router obtains the route from the frontend and pinpoints the necessary backend controller in order to do what the user is requesting. In addition, while the user is viewing or booking any trips, the router will then call the trips Controller. This means that the backend controller engages with the MongoDB database while using Mongoose to obtain or manipulate any data that is relevant to the user request, like for example the trip information or view packages.

When the MongoDB is able to process the query then it will give the controllers the requested results, then it will send this data to the HTTP, then the HTTP will give the results to the front end controller by doing the necessary rendering of the view with the new and updated information, while making a responsive and dynamic and pleasant user experience to the specific action the user request even if the user is signing in, booking trips or performing admin interactions.>

**Class Diagram**




<Travelerinfo: this option stores info about the star rating, location, room requested, price and etc.
Itinerary: this option stores info about the user's travel itinerary. It contains information like total price, total miles, and stopover about a trip etc.
CruiseInfo: this option stores the details about the cruise trips, as well as the ship's name, cabin type, and price.
Flightinfo: this one stores the details about the flight like name, seat class and price.
Tripinfo: stores the details about the trips as well is a parent class that encapsulates information about basic trip information that includes starting date, returning date, origin, and destination.
Hotelinfo: this one store information about the hotels, as well as the hotel's name, stars, location, the requested room and price. This one as well is also has a relationship with the HotelBooking and TravelAgent classes, this class is part of the TripInfo class.
HotelBooking: this option manages hotel booking for travelers.
CruiseBooking: this option manages cruise booking for travelers.
FlightBooking: this option manages flight booking for travelers.
Membership_Admin: this option allows administrative functionality to manage the user memberships and administrative privileges.
MemberAccount: this option stores info about member numbers, frequent flyer airline, status, and club.
Travel_Agent: this class allows the user or any other allowed entity responsible for bookings and interactions between travelers and the travel services.>

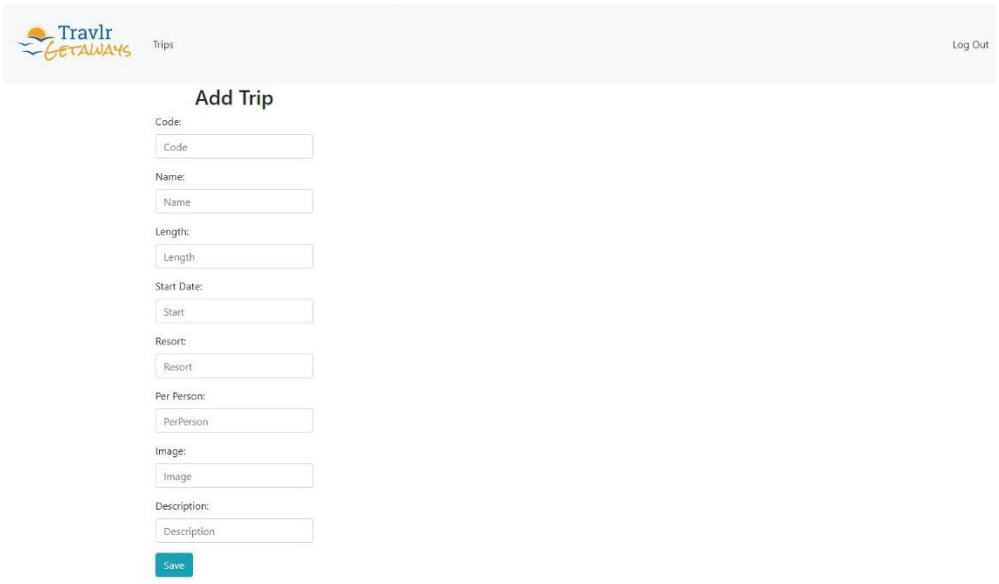
**API Endpoints**

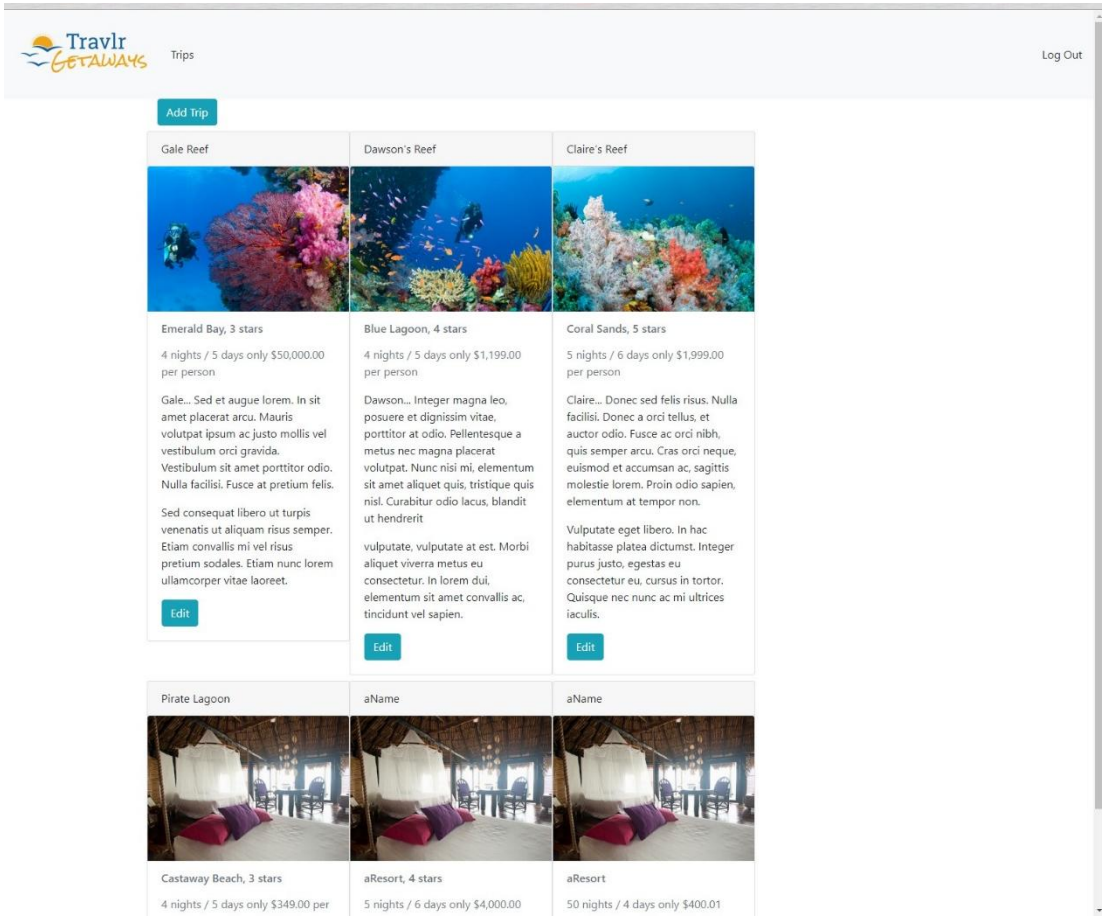
**API Endpoints**

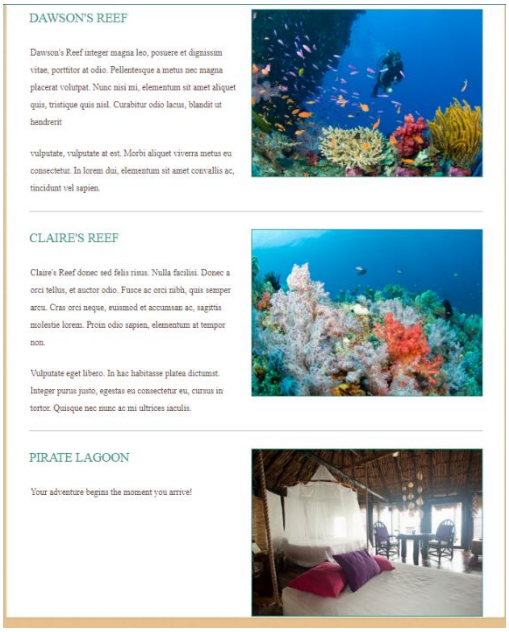
| Method | Purpose | URL | Notes |
|---|---|---|---|
| **GET** | <Retrieve list of things> | </api/things> | <Returns all active things> |
| **GET** | <Retrieve single thing> | </api/things/:thingId> | <Returns single thing instance, identified by the thing ID passed on the request URL> |
| **POST** | <Create a new list of things> | </api/things> | <Creates a new list of things> |
| **POST** | <Create a single thing> | </api/things/:thingId> | <Creates a single, identified by the thing ID passed on the request URL > |
| **PUT** | <Update an entire list of things> | </api/things> | <Updates and replaces an already existing list of things. All data from the existing list will be removed.> |
| **PUT** | <Update a single thing> | </api/things/:thingId> | <Updates and replaces an already existing thing. All data from the existing thing will be removed.> |
| **PATCH** | <Modify a list of things> | </api/things> | <Updates an existing list of things. This will not remove the previous list. It will only make the requested changes.> |
| **PATCH** | <Modify a single thing> | </api/things/:thingId> | <Updates an existing thing. This will not remove the previous list. It will only make the requested changes and modify the thing.> |
| **DELETE** | <Delete everything in a list of things> | </api/things> | <Deletes an entire list of things.> |
| **DELETE** | <Delete a single thing> | </api/things/:thingId> | <Deletes a single thing, identified by the thing ID passed on the request URL> |

**The User Interface**

<Insert screenshots from the development of the SPA development to show the following: (1) a unique trip, added by you, (2) the Edit screen, and (3) the Update screen.>


Travlr Getaways
Trips
Log Out
Add Trip
Code:
Code
Name:
Name
Length:
Length
Start Date:
Start
Resort:
Resort
Per Person:
PerPerson
Image:
Image
Description:
Description
Save


Travlr Getaways
Trips
Log Out
Add Trip
Gale Reef
Dawson's Reef
Claire's Reef
Emerald Bay, 3 stars
4 nights / 5 days only $50,000.00 per person
Gale... Sed et augue lorem. In sit amet placerat arcu. Mauris volutpat ipsum ac justo mollis vel vestibulum orci gravida. Vestibulum sit amet porttitor odio. Nulla facilisi. Fusce at pretium felis.
Sed consequat libero ut turpis venenatis ut aliquam risus semper. Etiam convallis mi vel risus pretium sodales. Etiam nunc lorem ullamcorper vitae laoreet.
Edit
Blue Lagoon, 4 stars
4 nights / 5 days only $1,199.00 per person
Dawson... Integer magna leo, posuere et dignissim vitae, porttitor at odio. Pellentesque a metus nec magna placerat volutpat. Nunc nisi mi, elementum sit amet aliquet quis, tristique quis nisl. Curabitur odio lacus, blandit ut hendrerit
vulputate, vulputate at est. Morbi aliquet viverra metus eu consectetur. In lorem dui, elementum sit amet convallis ac, tincidunt vel sapien.
Edit
Coral Sands, 5 stars
5 nights / 6 days only $1,999.00 per person
Claire... Donec sed felis risus. Nulla facilisi. Donec a orci tellus, et auctor odio. Fusce ac orci nibh, quis semper arcu. Cras orci neque, euismod et accumsan ac, sagittis molestie lorem. Proin odio sapien, elementum at tempor non.
Vulputate eget libero. In hac habitasse platea dictumst. Integer purus justo, egestas eu consectetur eu, cursus in tortor. Quisque nec nunc ac mi ultrices iaculis.
Edit
Pirate Lagoon
aName
aName
Castaway Beach, 3 stars
4 nights / 5 days only $349.00 per
aResort, 4 stars
5 nights / 6 days only $4,000.00
aResort
50 nights / 4 days only $400.01




The Angular project structure and the Express project structure is different they both have similitudes and differences however the Angular is more of a standalone frontend framework and we usually use pre build packages so the system can have a more standardized structure however the Express structure is more about the server side framework or backend system, what express does is obtain request and then returns the requested action as well it uses the sessions so it can identify all the different visitors on all the pages this can make the server to work seamless. The data that is being provided from all the different systems like node, MongoDB and more allows angular to update the data as soon as possible after being changed. The rich functionality provided by the SPA compared to a simple web application interaction is to load single web document and update the data of that document without loading more pages, this means that it has a faster loading time, reduction in server load whenever they obtain data from the API. It has many capabilities because SPAs are able to give dynamic loading, real-time updates, this makes them suitable for interactive and responsive applications.

the process of testing to make sure the SPA is working with the API to GET and PUT data in the database is to put the URL linked to the application using the postman REST client, this will trigger the system to give you either an error or a data and the status code. By doing so I'm able to see and make changes in order to make the website to work, I had multiple errors while doing the website the first one was when I tried to connect to MongoDB I kept having a issue connecting to the db and importing the data I wanted then I was getting a 400 error when loading the website which it meant that it was a bad request I was able to fix it. I made sure to check one every time I made a update the clients page to ensure that everything was still working seamlessly.